CS 3600 Intro to Artificial Intelligence
Homework 4
K Nearest Neighbours and Gradient Boosted Trees

This homework is due at 11:59 pm on Sunday, 15th of February. It must be uploaded to Gradescope by then. No credit will be given for late submissions.

The Situation

You've joined the data science team at FitWell, a startup building the next generation of fitness and health tracking apps. The goal is simple but ambitious: use smartphone sensors (accelerometer + gyroscope) to automatically detect what activity a user is performing. This will help provide accurate calorie estimates, detect sedentary behavior and even assist in physical therapy monitoring.

Your team has collected data from volunteers who carried smartphones in their pockets while performing six different activities: Walking, Walking Upstairs, Walking Downstairs, Sitting, Standing and Laying.

Your task is to build and evaluate an activity recognition system using K-Nearest Neighbors (KNN). Because FitWell plans to scale to millions of users, you'll explore different approaches: building KNN from scratch, using standard libraries, using FAISS for efficient similarity search, and finally comparing against a strong alternative model: gradient boosted decision trees (XGBoost).

1 Data

Link: HAR Dataset

About the dataset:

- **Features**: 561 pre-processed measurements from smartphone accelerometers and gyroscopes.
- **Labels**: Six activity classes, more information of which can be found in:
  `UCI Har Dataset/activity_labels.txt`
- **Split**: 561 pre-processed measurements from smartphone accelerometers and gyroscopes.

Dataset Download Instructions

To run this project, you need to download the UCI HAR Dataset from the link provided above. Follow the steps below:

1. Download the dataset from the provided link.
2. Extract the contents of the dataset into the project directory.
3. Ensure that the directory structure matches the format shown below:

Project Directory Structure

```
NearestNeighbor/
|
+-- UCI HAR Dataset/
|   +-- test/
|   |   +-- subject_test.txt
|   |   +-- X_test.txt
|   |   +-- y_test.txt
|   |
|   +-- train/
|       +-- subject_train.txt
|       +-- X_train.txt
|       +-- y_train.txt
|
+-- utils.py
+-- part1_knn_scratch.py
+-- part2_sklearn.py
+-- part3_faiss.py
+-- part4_xgboost.py
```

Make sure your Python files and the dataset folders are placed exactly as shown above.

Loading the Data (NumPy → JAX)

You may use **NumPy to load the dataset files**, since the raw data is stored as plain text. Use `np.loadtxt` to read the files and then convert the resulting NumPy arrays to JAX arrays before running your KNN implementation.

2 Implementation

2.1 KNN from scratch

You've just joined the startup, and before trusting libraries, your manager wants you to build KNN from the ground up to ensure you really understand it.

Tasks:

- Write the `euclidean_distances()` function. Please compute pairwise Euclidean distances of the entire matrix instead of running a for loop for every dimension since that will increase the time complexity.
- Implement `predict()` function in `ScratchKNN` class which predicts labels for the test data.

  Hint: You might want to use `jax.numpy.argsort()` for the k closest distances here.
- Implement `majority_vote()` to decide on the class label for the test dataset.

  This finds the most common of all the k neighbours to the query value. If two or more classes tie, the closer label is chosen. `predict()` will use `majority_vote()` and hence it must be invoked from `predict()` in your code.
- Write the function `confusion_matrix_multiclass()` to compute the confusion matrix for a multiclass classification problem without using any inbuilt functions such as `sklearn.metrics.confusion_matrix`.

  A confusion matrix is a performance evaluation tool used to visualize how well a classification model performs by comparing predicted labels with the true labels.

  In a multiclass classification problem, the confusion matrix is an $N \times N$ table, where N is the number of classes. Each row represents the actual class while each column represents the predicted class. The diagonal elements indicate the number of correct predictions for each class while the off-diagonal elements show the misclassifications.

  For example, in a 3-class problem (Classes A, B, and C), the confusion matrix looks like:

	Predicted A	**Predicted B**	**Predicted C**
Actual A	TP_A	$FP_{B\to A}$	$FP_{C\to A}$
Actual B	$FP_{A\to B}$	TP_B	$FP_{C\to B}$
Actual C	$FP_{A\to C}$	$FP_{B\to C}$	TP_C

Table 1: Example of a 3-Class Confusion Matrix

- Calculate the overall accuracy of the classification model in the `accuracy()` function.
- Test your implementation on the test set of the dataset by implementing `main()` in `part1_knn_scratch.py`. The `main()` function should load the dataset. Then, for the values of $k = \{4, 6\}$, create an object of the `ScratchKNN` class, fit the classifier to the training data and test it on the test data. Display the confusion matrix and accuracy using the `display_confusion_matrix_and_accuracy()` function which is already given. The `display_confusion_matrix_and_accuracy()` function already calls your implemented `confusion_matrix_multiclass()` and `accuracy()` functions so they do not need to be separately invoked.
- Report accuracy and confusion matrix for $k \in \{4, 6\}$.

Note: Speeding up KNN with `jax.jit` and vectorized JAX operations

Computing KNN distances is expensive because it involves large matrix operations. To make your implementation fast, we use JAX's Just-In-Time compiler:

- `jax.jit` compiles a function into optimized XLA code. The **first call** may be slower due to compilation, but later calls are much faster.
- In this homework, the KNN prediction is performed inside a JIT-compiled helper function: `predict_knn_jit(...)`.
- The value of k is treated as a **static argument** (so JAX can compile specialized code per k). You will see a decorator like: `@functools.partial(jax.jit, static_argnames=("k",))`

JAX / `jnp` functions you are expected to use

To keep your code fast and fully JIT-compatible, your implementation should use **JAX operations only** (no Python loops over points or dimensions). The following functions will be especially helpful:

- `jnp.sum(..., axis=..., keepdims=True)`: compute row-wise squared norms for the distance formula.
- `a @ b.T`: matrix multiplication to compute cross terms efficiently.
- `jnp.argpartition(dists, kth=k-1, axis=1)[:, :k]`: find indices of the k nearest neighbors efficiently (unordered).
- `jnp.argsort(dists, axis=1)[:, :k]`: alternative approach that fully sorts distances (typically slower than `argpartition`).
- `jnp.take_along_axis(dists, idx, axis=1)`: gather the k neighbor distances corresponding to chosen indices.
- Advanced indexing: `y_train[idx]` to gather neighbor labels.
- `jax.vmap(fn)`: vectorize your voting function across all test points (e.g., `jax.vmap(majority_vote)`).

Important: Avoid Python `for` loops over test points or training points. Use vectorized JAX operations so that `jax.jit` can compile and accelerate your code.

Check your implementation using the command: `python part1_knn_scratch.py`

2.2 Part 2: KNN using scikit-learn + Cross-Validation (Model Selection)

Now that you understand the mechanics, you can use scikit-learn to run experiments faster. Scikit Learn is a very popular library in Python that has a lot of ready-to-use functions for regression, classification, clustering and more.

In this part, you will:

1. Run scikit-learn KNN on the official train/test split and explore accuracy across different values of k.

2. Use **cross-validation on the training set** to select the best k (model selection).
3. Train a final KNN model using the best k and evaluate it on the **test set**.

Part 2A: KNN test accuracy sweep for $k \in [1, 10]$

Tasks:

- Use `sklearn.neighbors.KNeighborsClassifier`.
- Train on the HAR training set and evaluate on the HAR test set.
- Compute and display accuracy and confusion matrix for each $k \in [1, 10]$.

In the `KNN_using_sklearn()` function, create a `KNeighborsClassifier` object with the given value of k. Then, train the classifier on the training data and test it on the testing data. Return the predicted outputs using the classifier and the accuracy of the classifier. For accuracy, use the `accuracy()` function in `utils.py`.

Next, add code to the `main()` function in `part2_sklearn.py`. First, load the train and test dataset. Then, for each $k \in [1, 10]$, call `KNN_using_sklearn()` and display the confusion matrix and accuracy using `display_confusion_matrix_and_accuracy()`.

In a list named `ks`, append the different values of k from 1 to 10 (both inclusive). In a list named `accs`, for every value of k, append the result of the `accuracy()` function from `utils.py` to the list. These lists will be used to generate the plot `accuracy_vs_k.png`.

Part 2B: Cross-validation on training set to select best k

Before rolling this out to users, you must tune the model properly by finding the best value of k.

In the K-Nearest Neighbors (KNN) algorithm, the choice of the parameter k plays a crucial role in determining model performance. A small k can overfit, while a very large k can underfit. Hence, it is essential to find the optimal k.

To determine the best k, use m-fold cross-validation on the **training set only**:

1. Split the training data into m equal folds. For this assignment, use $m = 6$.
2. For each candidate $k \in [1, 20]$:
   (a) Train KNN on $(m - 1)$ folds.
   (b) Evaluate accuracy on the remaining fold.
   (c) Repeat so each fold is used once as the validation fold.
3. Compute the mean validation accuracy for each k.

4. Select the k with the highest mean validation accuracy.

Implement a function `cross_validate_knn()` in `part2_sklearn.py` that performs this procedure and returns the mean cross-validation accuracy for a given k. You may use scikit-learn utilities (e.g., fold splitting) and `KNeighborsClassifier` for training.

In `main()`, loop through $k = 1$ to 20 and print the mean accuracy using:

```
print(f"k={k:2d}, Mean CV Accuracy={mean_acc*100:.2f}%")
```

Select the best k (highest mean accuracy) and print:

```
print(f"\n Best k from CV: {best_k} with accuracy={cv_results[best_k]*100:.2f}%")
```

Part 2C: Final evaluation on the test set using best k

Finally, train a scikit-learn KNN model on the full training set using the best k found by cross-validation. Then evaluate on the official test set and display the confusion matrix and accuracy using `display_confusion_matrix_and_accuracy()`.

Deliverable for Part 2: All code for Parts 2A–2C should be implemented in a single file: `part2_sklearn.py`.

Check your implementation using the command: `python part2_sklearn.py`

2.3 Part 3: Scaling KNN with FAISS using IVF-PQ

FitWell plans to deploy activity recognition on millions of daily logs from users. Exact KNN will be too slow and memory-intensive. To handle this scale, you'll use FAISS, a library for fast similarity search, to implement an Approximate Nearest Neighbor (ANN) version of KNN using IVF-PQ (Inverted File with Product Quantization).

Goal: Build a compressed FAISS index that partitions the feature space into coarse clusters via IVF and further compresses each vector using Product Quantization (PQ). This enables large-scale search with significant speedup and memory savings while maintaining good accuracy.

Tasks:

- Build a FAISS `IndexIVFPQ` index on the training feature vectors.
- Use an `IndexHNSWFlat` quantizer to obtain coarse partitions for the IVF index.
- Query FAISS to find the k nearest neighbors for each test vector.
- Use the provided `majority_vote()` function to determine the predicted class label from the retrieved neighbors.

Implement the function `faiss_knn_time_and_acc()` to return the time and accuracy of FAISS-based KNN classification. Ensure that the training and test data are of type `float32`, and store the dimensionality (`d`) of the training data for initializing the FAISS index.

Below are some helpful FAISS functions and parameters used in this task:

1. **`quantizer = faiss.IndexHNSWFlat(d, nlinks)`**
   - **Input:**
     - `d` – Dimensionality of feature vectors (number of columns in X_{train}).
     - `nlinks` - Number of links (neighbors) per node in the HNSW graph. For your implementation, use `nlinks = 32`.
   - **Output:**
     - Returns a hierarchical navigable small world (HNSW) quantizer used for coarse clustering in the IVF stage.
   - **Description:**
     - The quantizer determines how vectors are partitioned into clusters before product quantization. HNSW improves recall while keeping indexing efficient.
2. **`index = faiss.IndexIVFPQ(quantizer, d, nlist, m, nbits)`**
   - **Inputs:**
     - `quantizer` – The coarse-level quantizer (HNSW in this case).
     - `d` – Feature dimensionality (number of columns in X_{train}).
     - `nlist` – Number of coarse clusters. For your implementation, `nlist = 128`.
     - `m` – Number of subquantizers. This value must divide d evenly. Since, `d = 561`, use `m = 17` for your implementation.
     - `nbits` – Number of bits per subquantizer. For your implementation, `nbits = 8`.
   - **Output:**
     - Returns an IndexIVFPQ object that stores compressed representations of vectors.
   - **Description:**
     - The IVF component partitions the dataset into *nlist* clusters.
     - The PQ component encodes each vector using m subquantizers, each using 8 bits.
     - Together, they enable efficient approximate nearest neighbor search.
3. **`index.train(X_train)`**
   - **Input:**
     - `X_train` – Training feature matrix used to learn cluster centroids and PQ codebooks.
   - **Output:** None
   - **Description:**
     - Before adding data, the IVF-PQ index must be trained to learn internal representations.
4. **`index.add(X_train)`**

- **Description:** Adds all training samples to the trained FAISS index. These become searchable entries.

5. `index.nprobe`

   - **Description:** Sets how many coarse clusters to probe during search. Larger values improve recall but increase latency. For your implementation, set `index.nprobe` to the value 10.

6. `D, I = index.search(X_test, k)`

   - **Inputs:**
     - `X_test` – Test data in `float32` format.
     - `k` – Number of nearest neighbors to retrieve.
   - **Outputs:**
     - `D` – Squared distances to the retrieved neighbors.
     - `I` – Indices of the nearest training vectors.
   - **Description:** Performs approximate nearest neighbor search on the compressed index.

After retrieving the neighbors, apply the provided `majority_vote()` function on each set of neighbor labels to decide the final predicted class.

In the function `scratch_knn_time_and_acc()`, use your earlier implemented `ScratchKNN` class to compute exact KNN accuracy and timing for comparison.

To test your implementation: `python part3_faiss.py`

NOTE: You might get the following warnings on executing the code, you may ignore it:
`WARNING clustering 7352 points to 256 centroids: please provide at least 9984 training points`

Expected Outcome: Compare FAISS and scratch KNN results. FAISS should yield a speedup with minimal loss in accuracy due to approximate search using IVF-PQ.

Note on Randomness in JAX (PRNG Keys)

JAX random number generation is **explicit and functional**: instead of using a global random state (like NumPy), you create a PRNG key and **split it** each time you need new randomness. If you generate multiple random values (or shuffle multiple times), you should keep splitting the key so each draw is reproducible and independent.

Example: JAX PRNG Key + Repeated Splitting

```
import jax
import jax.random as jr
import jax.numpy as jnp

key = jr.PRNGKey(0)  # fixed seed for reproducibility

# Example: generate multiple random arrays by splitting key each time
for _ in range(3):
    key, subkey = jr.split(key)
    r = jr.normal(subkey, shape=(5,))  # 5 random numbers
    print(r)
```

2.4 Part 4: Gradient Boosted Trees with XGBoost

KNN is a strong baseline, but it can be expensive at inference time because it must compare each test point to many training points. In this part, you will train a **gradient boosted decision tree** model using **XGBoost** and evaluate it on the HAR dataset.

Goal: Train an XGBoost multiclass classifier on the HAR training set and evaluate it on the HAR test set. You will then compare its results to the best KNN model you obtained using cross-validation in Part 2.

What to implement

You will work in `part4_xgboost.py`. Only implement the function:

- `xgb_train_and_predict(X_train, y_train, X_test, y_test)`

This function must:

1. Train an `xgboost.XGBClassifier` model on `(X_train, y_train)`.
2. Predict labels for `X_test`.
3. Return `(y_pred, test_acc)`, where `test_acc` is computed using `accuracy()` from `utils.py`.

The provided `main()` will print the test accuracy and display the confusion matrix.

Important note

- **XGBoost expects labels in the range** $\{0, 1, \ldots, C-1\}$ for multiclass classification. If your labels are $\{1, \ldots, 6\}$, shift them to $\{0, \ldots, 5\}$ during training and shift predictions back before reporting results.

Important XGBoost hyperparameters (what they mean)

Below are some of the key hyperparameters for `XGBClassifier`. You are encouraged to experiment with the values.

- `objective="multi:softmax"`: performs multiclass classification and outputs the predicted class label directly.
- `num class`: the number of activity classes (must match the number of unique labels).
- `n estimators`: number of boosted trees (more trees can improve accuracy but increases training time).
- `max depth`: maximum depth of each tree (higher = more complex model, higher overfitting risk).
- `learning rate`: step size for each boosting update (smaller often needs more trees; larger can overfit).
- `reg lambda`: L2 regularization strength (higher values generally reduce overfitting).
- `eval metric="mlogloss"`: metric used internally during training to track performance (multiclass log-loss; lower is better).
- `tree method="hist"`: uses a faster histogram-based algorithm (recommended for speed on CPU).

Accuracy target: Your final XGBoost test accuracy should be **at least 93%**. You may need to tune hyperparameters to reach this threshold.

Check your implementation using: `python part4 xgboost.py`

3 Deliverables

Submit the following files on Gradescope:

- `utils.py`
- `part1 knn scratch.py`
- `part2 sklearn.py`
- `part3 faiss.py`
- `part4 xgboost.py`